



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Lab Report on,

Retrieving Data from Two Different Tables

Course: Database Management System Lab

Course Code: CSE 2204

Lab Report No: 04

Submission Date: 17/11/2024

Submitted by	Submitted to
Adnan Zaman Niloy Student ID: 210142 Year: 2 nd Semester: 2 nd Session: 2021-22	Md Kamrul Islam, Associate Professor, Department of Computer Science and Engineering, Jashore University of Science and Technology

Experiment Name: Retrieving Data from Two Different Tables

Introduction:

In database management, retrieving data from multiple tables is a common requirement. SQL provides tools like JOIN, UNION, and subqueries to efficiently combine and filter data from different tables. This experiment demonstrates how to retrieve and merge relevant data from two tables using SQL commands.

Methodology:

1. Database Setup:

- Two tables were designed to simulate real-world scenarios:
 - Students Table:** Contains information about students and the courses they are enrolled in.
Schema: StudentID (Primary Key), Name, CourseID (Foreign Key).
 - Courses Table:** Contains details of the courses offered, including the instructor information.
Schema: CourseID (Primary Key), CourseName, Instructor.

Sample Data Inserted:

Students Table:

StudentID	Name	CourseID
1	Alice	101
2	Bob	102
3	Charlie	NULL

Courses Table:

CourseID	CourseName	Instructor
101	Mathematics	Dr. Smith
102	Physics	Dr. Jones

1. SQL Commands Used:

- **INNER JOIN:** To fetch only matching records from both tables based on CourseID.
- **LEFT JOIN:** To retrieve all records from the Students table, including unmatched ones.
- **RIGHT JOIN:** To retrieve all records from the Course table, including unmatched ones.
- **Subqueries:** To filter data based on conditions derived from related tables.
- **UNION:** To combine results from both tables into a unified dataset.

2. Steps Followed:

- a. Create tables Students and Courses and insert the sample data.
- b. Use INNER JOIN to retrieve student names with their corresponding course names.
- c. Use LEFT JOIN to list all students, even those not enrolled in any courses.
- d. Use RIGHT JOIN to ensure all courses are listed, even those without any students enrolled.
- e. Apply a subquery to identify students enrolled in courses taught by a specific instructor.
- f. Use UNION to merge names of students and instructors into a single list.

Result & Discussion:

1. INNER JOIN Query:

```
SELECT Students.Name, Courses.CourseName
FROM Students
INNER JOIN Courses ON Students.CourseID = Courses.CourseID;
```

Name	CourseName
Alice	Mathematics
Bob	Physics

Interpretation: Only students who are enrolled in courses are displayed.

2. LEFT JOIN Query:

```
SELECT Students.Name, Courses.CourseName
```

```
FROM Students
```

```
LEFT JOIN Courses ON Students.CourseID = Courses.CourseID;
```

Name	CourseName
Alice	Mathematics
Bob	Physics
Charlie	NULL

Interpretation: The query includes all students, showing NULL for those without a matching course.

3.RIGHT JOIN Query:

```
SELECT Students.Name AS StudentName, Courses.CourseName
```

```
FROM Students
```

```
RIGHT JOIN Courses ON Students.CourseID = Courses.CourseID;
```

StudentName	CourseName
Alice	Mathematics
Bob	Physics
NULL	Chemistry

4.Subquery :

SELECT Name

FROM Students

WHERE CourseID = (SELECT CourseID FROM Courses WHERE Instructor = 'Dr. Smith');

Name
Alice

Interpretation: Only students enrolled in courses taught by 'Dr. Smith' are listed.

5. UNION Query:

SELECT Name FROM Students

UNION

SELECT Instructor AS Name FROM Courses;

Name
Alice
Bob
Charlie
Dr. Smith
Dr. Jones

Interpretation: The query combines student and instructor names into a single list, removing duplicates.

Discussion:

- **INNER JOIN:** Effective when only matched records are needed, but it excludes unmatched data.
- **LEFT JOIN:** Ensures data completeness from the primary table, making it ideal for comprehensive reports.
- **Subqueries:** Useful for dynamic filtering based on conditions, though they may be less efficient compared to joins in some cases.

- **UNION:** Helps consolidate results but requires datasets to have the same structure. Duplicates are removed unless UNION ALL is used.
- **Efficiency Considerations:** Proper indexing on CourseID significantly improved query performance, especially for JOIN operations.

Conclusion:

This experiment highlighted the versatility of SQL in combining and retrieving data from multiple tables. Each technique—JOIN, subqueries, and UNION—has specific use cases, depending on the query's objectives and data relationships.

- JOIN operations are the backbone for merging related datasets efficiently.
- LEFT JOIN is invaluable when working with potentially incomplete relationships.
- Subqueries offer granular control but should be used judiciously in performance-critical environments.
- UNION is excellent for aggregating datasets but requires careful schema alignment.

Understanding these methods is crucial for crafting robust and efficient SQL queries to handle real-world database challenges. Proper indexing and optimized queries ensure scalability as dataset sizes grow.